Chapter 1 exercises

Keep units, formulas, baselines, and necessary assumptions. Identify missing information before comparing designs that are not directly comparable.

CH01-Q01 ISA and microarchitecture

A new processor implements the old ISA but changes a five-stage pipeline to eight stages, enlarges L2, and adds a new branch predictor.

1. Which properties does the ISA guarantee?
2. Which properties may change?
3. Why does compatibility not guarantee the same program execution time?

CH01-Q02 Performance decomposition

Workload A executes 3.6×10^8 instructions on P at CPI 1.50 and 2.4 GHz. Another compiler produces 3.0×10^8 instructions for Q at CPI 1.80 and 2.8 GHz.

1. Compute cycles using $IC \times CPI$, then CPU time using frequency.
2. Use P as the baseline to compute Q's speedup and percentage time reduction.
3. If Q's CPI rises to 2.10 with other inputs fixed, is Q still faster?
4. If Q instead runs at 2.4 GHz and CPI 1.40, recompute speedup. Which equation factors do the compiler and microarchitecture change?

CH01-Q03 Weighted CPI and instruction count

Class	Instructions	CPI
Integer	2.0×10^9	1
Load/store	5.0×10^8	4
Branch	5.0×10^8	3

1. Find total instructions, cycles, average CPI, and CPU time at 3 GHz. Reduce load/store CPI to 3 and compute speedup.
2. A statement has the following two implementations. Classes A/B/C have CPIs 1/2/3. Find each sequence's instruction count, cycles, and CPI. Which is faster at a common frequency, and why can more instructions win?

Sequence	A	B	C
S1	2	1	2
S2	4	1	1

Chapter 1 exercises (continued)

3. Which simple metric reverses their actual ranking?

CH01-Q04 Amdahl targets and impossibility

Computation occupies 72% of baseline service time; the remaining 28% is fixed. Computation becomes six times as fast.

1. Find normalized new time and speedup.
2. Find the limit if computation becomes infinitely fast.
3. Can any finite local speedup attain that limit? Distinguish a limiting bound from an attained value.
4. Another program takes 100 s, with 80 s improvable and 20 s fixed. Does a finite local speedup s achieve a 20 s total? Find s for a 50 s total.

CH01-Q05 Latency and throughput

An image processor handles one frame at a time without overlap and has 20 ms latency. A three-stage pipeline has 24 ms frame latency but completes one frame every 8 ms in steady state.

1. Find each steady-state throughput in frames/s.
2. When do the first and second pipelined frames complete?
3. Does the change help an application processing only one frame and prioritizing its completion time?

CH01-Q06 SPEC ratios and geometric means

Benchmark	X	Y
R1	1.44	1.20
R2	1.00	1.25
R3	0.81	1.08

1. Compute each geometric mean.
2. Which summary score is higher?
3. A different reference machine has arbitrary per-program times $T^{\star}_{\mathrm{ref},i}$. Prove directly that GM_X/GM_Y is unchanged. Does the arithmetic mean have this property?
4. How should a workload consisting almost entirely of R1 choose? Why is the summary insufficient?

Chapter 1 exercises (continued)

CH01-Q07 Power and energy

A processor consumes 40 W dynamic power during a 500 ms phase. In a first-order model, voltage and frequency both become 0.8 of their old values, switched capacitance stays fixed, and the phase's cycle count stays fixed.

1. Find new dynamic power.
2. Find new phase time.
3. Find energy and percentage energy reduction.

CH01-Q08 Performance pitfalls

A team speeds up a parser occupying 15% of baseline time by a factor of three. Server requests/s barely changes. CPU utilization rises and memory bandwidth is near saturation.

1. Use Amdahl's law to bound the optimistic latency speedup.
2. Why may actual throughput remain almost unchanged?
3. Which two categories of resource measurements should come next?

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study